

Vibecode and Secure an AI Agent Lifecycle with Antigravity and TDD

About this codelab

subject Last updated Jun 18, 2026

account_circle Written by Aron Eidelman, Sita Lakshmi Sangameswaran

1. Introduction

This lab is part of Kaggle's **5-Day AI Agents: Intensive Vibe Coding Course with Google**. You can find the additional codelabs and resources available on the [event site](#).

In this codelab, you will build a retail web application with an integrated AI Shopping Assistant using **Google's Agent Development Kit (ADK)**. You will use **Google Antigravity IDE** (Google's agentic IDE) to establish a secure, test-driven development (TDD) workflow.

Rather than treating security as a late-stage gate, you will learn to "shift security left" to the point of code inception. You will enforce development standards, automate [STRIDE threat modeling](#), and gate agent actions with git pre-commit hooks and agent-specific execution hooks.

What you'll do

- Scaffold and build a ADK 2.0 shopping assistant agent using Antigravity IDE and `agents-cli`.
- Set up project-level secure coding standards using a persistent context file (`CONTEXT.md`).
- Build and invoke a custom workspace-level STRIDE Threat Modeling Skill in Antigravity IDE.
- Enforce security guardrails directly during the TDD Plan Phase.
- Write outcome-based security tests in Pytest using Antigravity.
- Configure Git pre-commit hooks to automate [Semgrep](#) scans with local remediation loops in Antigravity.

What you'll need

- A web browser such as [Chrome](#)
- Familiarity with Python, Pytest, and basic terminal commands
- **Google Antigravity IDE** installed. See the [official website](#).
- **uv** package manager installed. See [uv installation guide](#).
- **Git** command line tool installed. Git is used exclusively for local version control in this lab, so a GitHub account is not required. See the [Git installation guide](#).

This codelab is for developers of all levels, including beginners. The entire lab will take approximately 60 minutes to complete.

Set up Authentication & Environment

Local Concept Demonstration: Because this lab focuses entirely on demonstrating local

development guardrails and pre-commit remediation loops, everything runs locally on your workstation. There is no need to authenticate to Google Cloud or connect your repository to GitHub!

Provide your authentication credentials for the agent to call Gemini models. Obtain a standard Gemini API key from [Google AI Studio](#) and export it in your IDE terminal session:

```
export GEMINI_API_KEY="your_api_key_here"
export GOOGLE_GENAI_USE_ENTERPRISE=FALSE
```

2. Setup Workspace and Toolchain

First, initialize the project workspace and install the underlying agent management toolchain. To do that, we will prompt Antigravity IDE to automate the setup process.

Note for Antigravity users: During this codelab, Antigravity may generate implementation plans or display popups before executing commands or writing code. Be sure to review and **approve** these plans or popups to allow Antigravity to proceed with the tasks.

Quota Tip: If you run out of quota during testing or development, you can switch the model in Antigravity to another available model.

👉 **Prompt to Antigravity:**

Help me set up my local project workspace. Please:

1. Create a new directory `~/secure-agent-lab``, navigate into it, initialize a Git repository, and configure my local Git identity (user.name: `"Kaggle Student"`, user.email: `"student@example.com"`).
2. Create and activate a Python virtual environment using ``uv``.
3. Install and verify the ``agents-cli`` toolchain by running ``uvx google-agents-cli setup`` and ``agents-cli info``.

What to expect: Antigravity will execute the necessary terminal commands on your behalf, establishing a clean Git repository and installing the companion ADK skills into your IDE.

3. Scaffold the ADK Agent Project

In this phase, we guide Antigravity to use `agents-cli` to scaffold a fully functional ADK 2.0

agent project called `shopping-assistant`. We will instruct Antigravity to design a retail assistant with a discount redemption tool.

Note: To show the power of automated gating hooks, in this prompt, we explicitly instruct Antigravity to include a simulated vulnerability: a hardcoded mock API key used for local development.

👉 Prompt to Antigravity:

Use ``agents-cli`` to scaffold a new ADK 2.0 agent project called ``shopping-assistant``. The workflow should act as an AI shopping assistant for a retail store. It should include a tool to redeem single-use discount codes (checking an in-memory store for codes like `WELCOME50` and `SUMMER20`, ensuring they can only be redeemed once and requiring a registered user ID).

Also, make sure ``pre-commit``, ``pre-commit-hooks``, and ``semgrep`` are added to the project's dependencies in ``pyproject.toml`` and installed. Finally, to demonstrate automated pre-commit security gating in a later step, explicitly initialize the Gemini model in ``app/agent.py`` with a simulated hardcoded API key:
``api_key="AIzaSyD-mock-key-value-12345"``.

What to expect: Antigravity will execute `agents-cli scaffold create shopping-assistant --adk`, configure `pyproject.toml`, and implement the agent logic.

[4. Explore the Agent Code](#)

👉 Ask Antigravity to explain the generated code:

Read and explain the project structure of my new shopping-

assistant agent.

Walk me through how ``app/agent.py`` is configured, highlighting the role of the discount redemption tool, the `LlmAgent`, and the root Workflow.

In the Antigravity IDE, newly created project files are displayed directly in the auxiliary pane (left side). You can view `shopping-assistant/app/agent.py` there or open it from the IDE file explorer.

Note: Because Antigravity generates implementations dynamically, the code in your `app/agent.py` file may be structured slightly differently than the snippet below, but it will share the same core ADK 2.0 concepts.

```
# shopping-assistant/app/agent.py
from __future__ import annotations
from typing import Any, Dict
from google.adk.agents.context import Context
from google.adk.apps.app import App
from google.adk.events.event import Event
from google.adk.workflow import Edge, Workflow
from google.adk.workflow.agents.llm_agent import LlmAgent
from google.adk.models.google_llm import Gemini
from google.adk.workflow.node import node
from pydantic import BaseModel, Field

# Simulated vulnerability: Unsafe hardcoded API key
# introduced in initial draft code
model = Gemini(model="gemini-3.1-flash-lite",
api_key="AIzaSyD-mock-key-value-12345")

# In-memory discount redemption store (simulating database
state)
DISCOUNT_STORE: Dict[str, bool] = {"WELCOME50": False,
"SUMMER20": False}

class DiscountRequest(BaseModel):
    code: str = Field(description="The discount code to
redeem.")
    user_id: str = Field(description="The ID of the user
requesting redemption.")
```

```

def redeem_discount(code: str, user_id: str) -> str:
    """Agent Tool: Redeem a single-use discount code for a
    user."""
    if code not in DISCOUNT_STORE:
        return "Error: Invalid discount code."
    if DISCOUNT_STORE[code]:
        return "Error: Discount code has already been
redeemed."
    if not user_id or user_id.startswith("guest_"):
        return "Error: Registered user account required to
redeem discounts."

    DISCOUNT_STORE[code] = True
    return f"Success: Discount code {code} redeemed
successfully for user {user_id}."

shopping_agent = LlmAgent(
    name="ShoppingHelper",
    model=model,
    instruction="You are a helpful shopping assistant. Use
your tools to redeem discount codes for users.",
    tools=[redeem_discount]
)

root_workflow = Workflow(
    name="shopping_assistant_workflow",
    edges=[*Edge.chain('START', shopping_agent)]
)

app = App(
    name="shopping_assistant",
    root_agent=root_workflow
)

```

Production Best Practice: Concurrency & Race Conditions

In our tool logic, we check an in-memory dictionary (`DISCOUNT_STORE`) and modify its state. In a multi-threaded production environment with a real database, two concurrent requests could both read the code as unredeemed before either write commits, causing a **double-redemption** vulnerability. In production, always use pessimistic locking (e.g., `.with_for_update()`) or optimistic versioning to ensure transaction isolation.

Lint the Initial Agent Graph

To verify that our newly scaffolded agent graph compiles correctly, we can prompt Antigravity to run the linter and test the agent on our behalf.

👉 **Prompt to Antigravity:**

Run ``agents-cli lint`` on our ``shopping-assistant`` project to verify syntax and refactor if any issues.

What to expect: Antigravity will execute `agents-cli lint` and `agents-cli lint --fix` to fix any formatting or lint issues detected.

5. Create Project-Specific Rules

To prevent overloading your agent's active memory with thousands of pages of generic security documentation—which causes context rot and reasoning latency, you must establish a "paved road" of pre-approved secure conventions. In Antigravity IDE, you can establish these guardrails by creating a persistent context file.

👉 **Prompt to Antigravity:**

Create a customization directory named ``shopping-assistant/.agents`` and create a file ``shopping-assistant/.agents/CONTEXT.md`` defining our secure coding standards:

```
# Local Project Context & Secure Coding Standards
```

```
## Core Paved Roads
```

```
We systematically address common vulnerability classes by  
guiding the agent  
to use our pre-configured, secure-by-default helper  
patterns instead of  
writing raw implementation logic from scratch.
```

```
1. **Tool Input Validation**: Every agent tool must
```

validate incoming
parameters against strict Pydantic schemas rather than
parsing raw
dictionaries or strings.

2. ****No Shell Execution****: Never use ``run_command`` or raw
shell execution

tools unless explicitly approved by ``hooks.json``.

3. ****Pre-Commit Remediation Loop****: If a git commit fails
due to a pre-commit

hook error (such as a Semgrep scan finding), you **MUST**
treat the violation

as a refactoring task, apply targeted fixes, run tests
to verify no

regressions, and attempt to commit again.

What to expect: Antigravity will create the `.agents/` directory and the `CONTEXT.md` file,
displaying it directly in the auxiliary pane for your review.

Caution: Failing to centralize these rules causes the agent to fall back on basic training data,
which might result in vulnerable string concatenation or open redirect vectors.

6. Configure Local Gating Hooks

To prevent insecure code or secrets from ever leaving your workstation, configure automated
gates at the boundaries of your development environment *before* committing code. We will
prompt Antigravity to generate the hook configurations, then install them using the terminal.

1. Git Pre-Commit Hook

To ensure our static analysis reliably catches simulated credentials and blocks insecure commits,
we will define a custom Semgrep rule and configure a Git pre-commit hook to enforce it.

Define a Custom Semgrep Rule

Default Semgrep rules (`--config auto`) do not flag mock keys containing words like
"mock" or hyphens due to low-confidence scoring. To reliably detect our hardcoded API key,
we will create a custom local rules file utilizing direct regex scanning instead of generic rules.

👉 **Prompt to Antigravity:**

Create a custom Semgrep rules file ``shopping-
assistant/.semgrep/rules.yaml``

with a rule to detect hardcoded Google API key prefixes (matching regex ``AIzaSy[A-Za-z0-9_\\-]*``). Configure it for Python files with an error severity and a clear security warning message.

What to expect: Antigravity will create `shopping-assistant/.semgrep/rules.yaml` defining our custom scanning rule. Review the generated rule definition:

```
# shopping-assistant/.semgrep/rules.yaml
rules:
  - id: detect-hardcoded-google-api-key
    pattern-regex: 'AIzaSy[A-Za-z0-9_\\-]*'
    message: "Security Issue: Hardcoded Google API key
prefix detected."
    languages:
      - python
    severity: ERROR
```

Configure the Pre-Commit Hook

Next, configure the Git pre-commit hook to run our custom Semgrep rule. When executing hooks in a subdirectory layout, paths to local configs must be relative to the Git repository root directory (`shopping-assistant/.semgrep/rules.yaml`), not the project subdirectory.

👉 Prompt to Antigravity:

Create a ``shopping-assistant/.pre-commit-config.yaml`` file configured to run local pre-commit hooks (end-of-file-fixer, trailing-whitespace) and a local Semgrep security scan on commit for Python files. Ensure Semgrep is configured with the ``--error`` flag and references our custom rules file relative to the Git repository root. Once created, run ``pre-commit install`` in the terminal to activate the hooks.

What to expect: Antigravity will generate `shopping-assistant/.pre-commit-`

`config.yaml` and execute `pre-commit install` on your behalf. Review the generated configuration:

```
# shopping-assistant/.pre-commit-config.yaml
repos:
  - repo: local
    hooks:
      - id: end-of-file-fixer
        name: End of File Fixer
        entry: end-of-file-fixer
        language: system
        types: [file]
      - id: trailing-whitespace
        name: Trailing Whitespace
        entry: trailing-whitespace-fixer
        language: system
        types: [file]
      - id: semgrep
        name: Semgrep Security Scan
        entry: semgrep --error --config shopping-
assistant/.semgrep/rules.yaml
        language: system
        types: [python]
```

Tip: The `--error` flag is **critical**; without it, Semgrep will log the security finding to standard output but return exit code 0 (success by default), which allows the unsafe commit to pass. Pre-commit only blocks commits on non-zero exit codes.

This configuration ensures that even if an agent is running in fully autonomous, non-interactive mode, the pre-commit gate will fire and block any commit that fails static analysis scans.

Direct Execution Commands (For Manual Verification)

To verify your static analysis setup manually without triggering a full commit hook cycle, you can execute these checks directly from the terminal.

Via pre-commit (from the "shopping-assistant" directory):

```
uv run pre-commit run semgrep --all-files
```

Via Semgrep directly (from the "shopping-assistant" directory):

```
uv run semgrep --error --config .semgrep/rules.yaml app/agent.py
```

2. Built-in Antigravity Agent Hook

For deeper, mid-trajectory gating, configure an agent hook in `shopping-assistant/.agents/hooks.json`. Unlike git hooks, agent hooks intercept Antigravity *before* it can run critical tools (like executing shell commands) on your system.

👉 Prompt to Antigravity:

Create a ``shopping-assistant/.agents/hooks.json`` file configured with a ``PreToolUse`` hook that intercepts ``run_command`` executions and runs ``python3 .agents/scripts/validate_tool_call.py`` with a 10-second timeout.

What to expect: Antigravity will create `shopping-assistant/.agents/hooks.json`. Review the generated configuration:

```
{
  "enabled": true,
  "PreToolUse": [
    {
      "matcher": "run_command",
      "command": "python3 .agents/scripts/validate_tool_call.py",
      "timeout": 10
    }
  ]
}
```

Caution: To safely intercept tool executions like `run_command`, the `PreToolUse` event definition **MUST** include the mandatory `matcher` field. Leaving this out or misconfiguring it can let the agent execute raw, unchecked shell actions.

Next, prompt Antigravity to create the underlying tool validation script referenced by our hook.

👉 Prompt to Antigravity:

Create the script ``shopping-assistant/.agents/scripts/validate_tool_call.py`` with logic to inspect ``run_command`` executions passed using `stdin` and block destructive commands like ``rm -rf /``.
What to expect: Antigravity will generate `shopping-assistant/.agents/scripts/validate_tool_call.py`, similar to the one below. Review the generated validation script:

```
# shopping-assistant/.agents/scripts/validate_tool_call.py
import sys
import json

def main():
    try:
        context = json.load(sys.stdin)
        command = context.get("tool_args",
                               {}).get("CommandLine", "")

        if "rm -rf /" in command or "mkfs" in command:
            print("BLOCKED: Destructive command detected.",
                  file=sys.stderr)
            sys.exit(1)

        print("APPROVED: Command validation passed.")
        sys.exit(0)
    except Exception as e:
        print(f"Validation error: {e}", file=sys.stderr)
        sys.exit(1)

if __name__ == "__main__":
    main()
```

Hook Trade-offs

-
- **Git Hooks:** Native to version control; run even if an agent runs in fully

autonomous, non-interactive mode. However, they can be bypassed using the `--no-verify` flag during a commit.

- **Agent Hooks:** Capture events mid-trajectory to block dangerous tool commands, but they do not protect the repository if a developer bypasses the IDE.

Local gates reinforce good developer habits and help catch vulnerabilities instantly. Remember, however, that local gates can be bypassed, meaning your remote, isolated CI/CD pipeline remains your ultimate, unbyassable security barrier.

7. Implement STRIDE Threat Modeling Skill

Now, you will grant Antigravity the specialized knowledge of a security architect. Antigravity skills are modular, declarative Markdown directories that instruct the agent how to run multi-step reasoning jobs. Antigravity automatically discovers any skills placed in `.agents/skills/`.

👉 **Prompt to Antigravity:**

```
Create a local skill directory
`shopping-assistant/.agents/skills/stride-threat-model/`
and create a skill
definition file `shopping-assistant/.agents/skills/stride-
threat-model/SKILL.md`
with the following content:
```

```
---
name: stride-threat-model
description: Performs a systematic STRIDE threat modeling
assessment on the
current project's codebase and architecture. Use this when
starting a new
implementation phase or reviewing existing components.
---
```

```
# STRIDE Threat Modeling Skill
```

```
## Goal
```

```
Guide the agent to analyze the workspace directory
structure, configuration
files, and code files to produce a structured
`threat_model.md` assessment.
```

Instructions

1. ****Analyze System Boundaries****: Map the entry points (tools, workflows, prompts) and data storage layers.
2. ****STRIDE Evaluation****: Evaluate the system against the six STRIDE pillars:
 - ****Spoofing****: Are caller identity boundaries verified before executing sensitive tool logic?
 - ****Tampering****: Can users manipulate data flows, parameters, or underlying state?
 - ****Repudiation****: Are critical transactions securely logged?
 - ****Information Disclosure****: Are we risking leakage of PII, internal tokens, or raw stack traces?
 - ****Denial of Service****: Are there rate limits on expensive database or LLM queries?
 - ****Elevation of Privilege****: Can an unauthenticated user bypass access control to reach privileged tool actions?
3. ****Output****: Generate a highly structured ``threat_model.md`` saved directly into the workspace root.

What to expect: Antigravity will create the custom skill directory and `SKILL.md` file.

Now, let's execute your newly created skill to evaluate your active project graph (`shopping-assistant/app/agent.py`).

👉 **Prompt to Antigravity:**

Run `stride-threat-model` on our `shopping-assistant` agent graph.

What to expect: Antigravity matches your intent, loads the threat assessment instructions on-demand from your local skill directory, analyzes your existing `agent.py` file, and generates a structured `threat_model.md` directly in your `shopping-assistant` root. This approach keeps your everyday working context clean and lightweight while giving you instant access to expert security reasoning.

8. Gate the TDD Plan Phase

Before implementing further features or refactoring code, Antigravity utilizes its semantic codebase understanding to map out changes in an `implementation_plan.md` and a `task.md` checklist. To force Antigravity to design for security up-front, we configure a system rule in `.agents/CONTEXT.md` that restricts plan approval.

👉 Prompt to Antigravity:

Append the following TDD planning gate instruction to the bottom of

```
`shopping-assistant/.agents/CONTEXT.md`:
```

```
## TDD Planning Gate
```

```
During the Plan phase, you must decompose the workspace task into logical, modular stages. Every implementation plan MUST include a dedicated
```

```
**Security Boundaries & Assertions** section outlining specific edge cases that could exploit the feature.
```

```
What to expect: Antigravity will update shopping-assistant/.agents/CONTEXT.md with the new planning gate rule.
```

When you prompt Antigravity to build or refactor a feature in subsequent steps, it will automatically analyze this rule and present an implementation plan with an explicit security breakdown. You will be able to review this plan directly in Antigravity's interactive dialog or auxiliary pane, where you must click **Proceed** or **Approve** before code generation begins.

Test the Planning Gate (Optional)

To observe this planning gate in action right now, try prompting Antigravity with one of the following feature requests. Instead of immediately generating code, Antigravity will enter the Plan phase and present an `implementation_plan.md` featuring a detailed **Security Boundaries & Assertions** breakdown (e.g., identifying race conditions, unauthorized privilege escalation, or negative point values) for your review!

👉 Prompt to Antigravity (Choose one to test):

Plan a new agent tool ``award_loyalty_points`` that awards points to a user's account after a successful purchase.

Plan a new agent tool ``process_cart_checkout`` that receives a cart ID and discount code, applies the discount, and processes the order.

Plan a new agent tool ``update_discount_status`` that allows administrators to activate or deactivate discount codes in the store.

9. Write Isolated, Outcome-Based Tests

Now we guide Antigravity to write comprehensive security tests for our existing ADK agent tools. To ensure our security tests are resilient and realistic, we structure them based on two core principles:

- **Assert on outcomes, not interactions:** Assert on final return strings and state mutations instead of writing fragile mocks that spy on internal helper calls.
- **Enforce Strict Guardrails:** Verify that tools enforce explicit business logic boundaries (like single-use redemption and registered user rules).

👉 **Prompt to Antigravity:**

Use ``agents-cli`` and `pytest` to generate an outcome-based security test suite in ``shopping-assistant/tests/test_agent.py``. Inspect ``app/agent.py`` and write tests to verify all security boundaries and business logic guardrails for the ``redeem_discount`` tool.

What to expect: Antigravity will generate `shopping-assistant/tests/test_agent.py`, similar to the one below. Review the generated test file:

```
# shopping-assistant/tests/test_agent.py
import pytest
```

```

from app.agent import redeem_discount, DISCOUNT_STORE

@pytest.fixture(autouse=True)
def reset_store():
    """Ensure strict test isolation by resetting in-memory
store state before each test run."""
    DISCOUNT_STORE["WELCOME50"] = False
    DISCOUNT_STORE["SUMMER20"] = False
    yield
    DISCOUNT_STORE["WELCOME50"] = False
    DISCOUNT_STORE["SUMMER20"] = False

def test_discount_code_can_only_be_redeemed_once():
    """Verify a user cannot submit a request reusing a
single-use code."""
    # First redemption - should succeed
    res_one = redeem_discount("WELCOME50", "user_123")
    assert "Success" in res_one
    assert DISCOUNT_STORE["WELCOME50"] is True

    # Second redemption trying to reuse the same code
    res_two = redeem_discount("WELCOME50", "user_456")
    assert "Error: Discount code has already been redeemed"
in res_two

def test_discount_redemption_rejects_invalid_code():
    """Verify that unknown discount codes are hard-
blocked."""
    res = redeem_discount("INVALID999", "user_123")
    assert "Error: Invalid discount code" in res

def test_discount_redemption_rejects_guest_accounts():
    """Verify that unauthenticated guest accounts cannot
redeem discounts."""
    res = redeem_discount("SUMMER20", "guest_999")
    assert "Error: Registered user account required" in res
    assert DISCOUNT_STORE["SUMMER20"] is False

```

Verify TDD GREEN Phase

To verify that our newly generated security tests pass successfully against our existing `agent.py` implementation, prompt AntigraVity to execute pytest on your behalf.

👉 Prompt to Antigravity:

Run ``uv run pytest tests/test_agent.py`` on our ``shopping-assistant`` project to verify that our security tests pass successfully.

What to expect: Antigravity will execute `uv run pytest tests/test_agent.py` in the terminal. All test cases pass successfully. The logical application boundaries are now secure, but we must make sure our static scanner catches the hardcoded API key during commit.

10. Verify Gating and Agent Self-Correction

Now that tests are green, you enter the **Refactor and Commit Phase**. This is where our local security scans and pre-commit hooks verify that no repo leaves the workstation with vulnerabilities, and Antigravity demonstrates its autonomous self-correction capabilities.

Commit Hygiene: Do not use the `--no-verify` flag when executing the commit command! We specifically want the pre-commit hook to fire so you can observe Semgrep intercept the simulated API key leak.

```
1           1       In your terminal, navigate to the project directory and commit the code
                using uv run to ensure local pre-commit hook binaries are active on your PATH:

                cd ~/secure-agent-lab/shopping-assistant
2           2       git add .
3           3       uv run git commit -m "feat: implement shopping
                assistant agent"
4           4
5           5       Observe that the commit fails because the Git pre-commit hook runs
                Semgrep automatically:

                Semgrep Security
                Scan.....
                .Failed
6           6       - hookid: semgrep
7           7
8           8       app/agent.py
9           9       Security Issue: Hardcoded Google API key
                prefix detected.
10          10
11          11     Guided by the Pre-Commit Remediation Loop rule configured
                in .agents/CONTEXT.md, Antigravity automatically intercepts the pre-commit
```

failure in the IDE terminal, reads the Semgrep error logs, and starts a refactoring step to move from a hardcoded API key to retrieving the key securely.

12 12 Review the refactored code generated by Antigravity in `shopping-assistant/app/agent.py` to resolve the API key leak.

13 13 Antigravity automatically runs `pytest` to verify the tests are still green.

14 14 Antigravity attempts the commit again on your behalf. Provided your local Git identity is configured and a repo is initialized, the scan passes, and the commit succeeds!

This demonstrates the core power of combining TDD with local pre-commit hooks and agent loops. Instead of waiting for a remote CI/CD failure, the agent is kept accountable locally, refactoring its code to be secure-by-default before it is ever pushed.

11. Run and Test the Agent Locally

Now that the security boundaries are verified and committed, run the ADK agent locally using your exported Gemini API key to interact with it in the local playground.

1 1 In your terminal, ensure your Gemini API Key is exported in your environment:

```
echo $GEMINI_API_KEY # Verify your key is exported
```

2 2

3 3 Launch the local development playground for your agent:

```
cd ~/secure-agent-lab/shopping-assistant
```

4 4 `agents-cli playground`

5 5

You should see output indicating the local playground server is running on port 8080:

```
* Serving ADK Playground
```

6 6 * Running on `http://127.0.0.1:8080/dev-ui/?`

```
app=app
```

7 7

8 8 Open the provided URL in your web browser to start chatting with your Shopping Assistant agent interactively. Try asking it to redeem a discount code:

```
Can you redeem the discount code WELCOME50 for user
user_123?
```

9 9

The agent will process your request using Gemini model execution and attempt to redeem the discount. **Note:** Because Antigravity generates tool logic dynamically, your agent's implementation of the registered user check may vary. If the agent rejects `user_123` as unregistered (e.g., `User ID 'user_123' is not registered`), ask the agent which user IDs are registered in its store (or check `app/agent.py`), and retry

with a valid registered user ID!